

SLAM performance prediction

Mauro Tellaroli

Statistical Methods for Machine Learning

Università degli Studi di Milano

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Introduction	2
1.1	SLAM methods	2
1.2	Accuracy of SLAM methods	2
1.3	APE calculation	2
1.4	Project goal	3
2	Data	4
2.1	Dataset composition	4
2.2	Data preprocessing	4
2.3	Data augmentation	6
3	Model	6
4	AlexNet	6

1 Introduction

In the field of autonomous mobile robot, the Simultaneous Localization And Mapping (SLAM) represents one of the most significant problems. As the name suggests, the goal of the SLAM is incrementally obtain from a mobile robot a map of the environment and simultaneously locate the robot's position inside it. The difficulty of the SLAM problem is that to precisely localize itself, the robot needs an accurate map, while to obtain an accurate map, it needs an accurate localization inside the map. This type of problems is called chicken-and-egg problem.

1.1 SLAM methods

In the last 30 years several approaches have been developed to solve the SLAM problem leading to methods extensively used in both common and industrial scenarios. The main paradigms [4] used are:

- Extended Kalman Filter: a vector containing the estimates of the robot position and the environment landmark is used. In addition, a matrix composed of the correlations between the positions and landmark estimates is also used. Both the vector and the matrix are updated using the extended Kalman Filter.
- Particle Filter: the Monte Carlo method is used to sample the possible states of the robot. A random sample is called a particle and it's composed of the robot position and a map estimate. The main steps are: predict the robot's pose for each particle, update particle weights based on sensor data, resample particles according to their weights, and update each particle's map.
- Graph-based: a graph is used where nodes represent robot poses or landmarks, and edges represent spatial constraints derived from sensor observations or odometry. The SLAM problem is formulated as a graph optimization task, where the goal is to find the configuration of nodes that best satisfies all constraints, typically using nonlinear optimization techniques.

1.2 Accuracy of SLAM methods

The heterogeneity of SLAM methods makes it difficult to find a measure that can evaluate their goodness. Many approaches rely on manual evaluation or utilize information derived from the specific algorithm. Nowadays the most used metric is the Absolute Pose Error (APE)[1]. The APE measures the difference between the estimated trajectory produced by the SLAM algorithm and the ground truth trajectory, focusing on the global consistency of the estimated poses. The big advantage of the APE is that it allows for a quantitative assessment of how well the SLAM algorithm reconstructs the robot's trajectory over time, independent of the internal workings of the specific method used.

The APE is composed of two sub-metrics:

1. The Absolute Translational Error (ATE): it measures the difference in position, typically using the Euclidean distance between corresponding poses.
2. The Absolute Rotational Error (ARE): it measures the difference in orientation, often computed as the angular distance between corresponding rotations.

Figure 1 and 2 show respectively the ATE variation over time and the difference between the ground truth and the robot estimated trajectory during an environment exploration.

1.3 APE calculation

As anticipated in the previous section, the APE metric is based on the difference between the estimated robot poses and the corresponding ground truth poses.

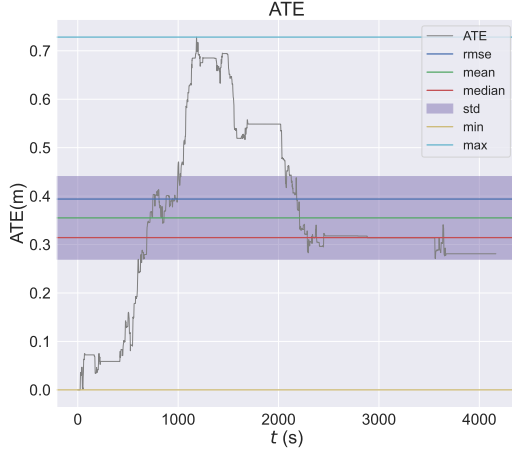


Figure 1: ATE variation over time

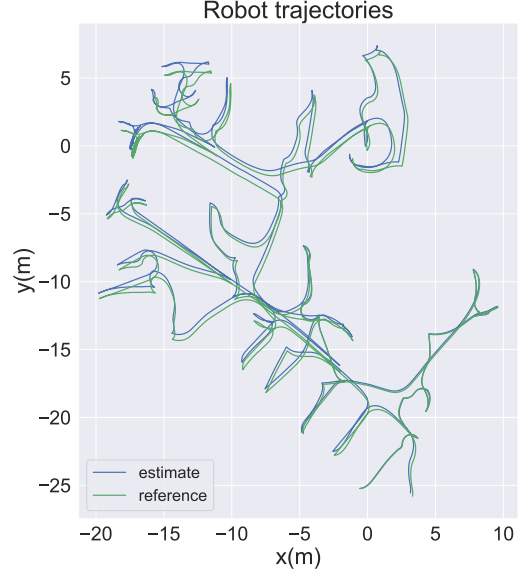


Figure 2: Difference between real (in green) and estimated (in blue) trajectory

Formally, let $\mathbf{x}_{1:T}$ be the poses of the robot estimated by a SLAM algorithm from time step 1 to T , and let $\mathbf{x}_{1:T}^*$ be the corresponding ground truth poses.

Let $\delta_{i,j}$ be the relative transformation that moves the robot from pose x_i to x_j :

$$\delta_{i,j} = x_j \ominus x_i$$

$$\delta_{i,j}^* = x_j^* \ominus x_i^*$$

Let $trans(\cdot) \in \mathbb{R}^2$ be the function that extracts the translational (position) component from a transformation, and let $rot(\cdot) \in \mathbb{R}^2$ extract the rotational (orientation) component as Euler angle.

Finally, to compute the APE:

$$APE = \underbrace{\frac{1}{N} \sum_{i,j} trans(\delta_{i,j} \ominus \delta_{i,j}^*)^2}_{ATE} + \underbrace{\frac{1}{N} \sum_{i,j} rot(\delta_{i,j} \ominus \delta_{i,j}^*)^2}_{ARE}$$

1.4 Project goal

Typically, the APE is calculated a posteriori, after the robot has explored the entire environment. The main goal of this project is to propose a predictor for the SLAM performance, specifically the APE, before running the algorithm or during its execution, using features derived from the input data or intermediate results. This type of prediction can improve the robustness and efficiency of autonomous mobile systems since the position error (APE) can be used to correct the robot trajectory.

The work in [3] proposes two main APE predictors, one based on linear regression, and another one based on a Gaussian Process via an RBF Kernel. Although Gaussian Process regression showed slightly better performance compared to linear regression model, the authors preferred linear regression because they considered it a more interpretable model. In both cases, a feature extractor is used [2] obtaining a set of structural features to represent the environment as a vector, which is then passed to the model.

2 Data

2.1 Dataset composition

The dataset is composed of 4613 datapoints. Each datapoint is represented as a tuple (M, a, ate, are) , where:

1. $M \in [0, 255]^{n \times n}$: floorplan images representend as grayscale maps that a robot has obtained during its navigation. Some examples are reported in Figure 3;
2. $a \in \mathbb{R}^+$: area measurements of the floorplan images, expressed in m^2 ;
3. $ate \in \mathbb{R}^+$: ATE values, expressed in m ;
4. $are \in \mathbb{R}^+$: ARE values, expressed in rad ;

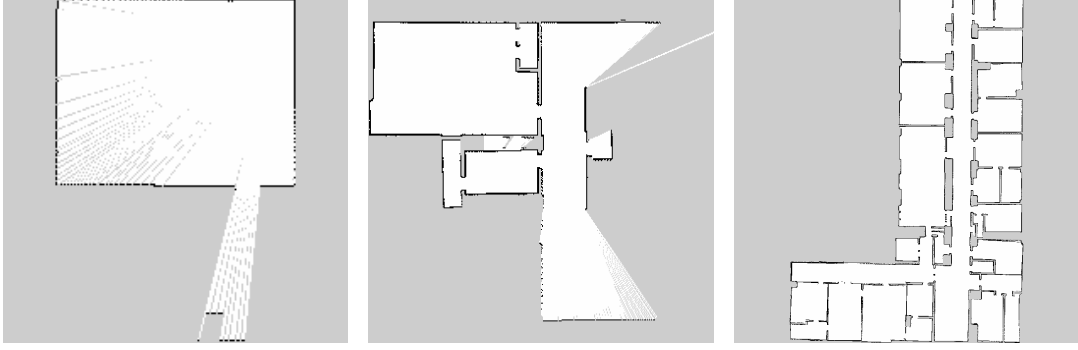


Figure 3: Examples of grayscale floorplan images used as input; the white pixels represent the free area, the black pixels the obstacles and the gray pixels the unknown area.

2.2 Data preprocessing

Evaluating the APE values shows that they are mainly distributed towards zero, with a significant presence of outliers, especially for ATE values. Figure 4 shows the box plot with highlighted the 0.99-quantile and the first value such that its Z-score is greater or equal to 3.5; the Z-score of a number x is $Z = \frac{x - \bar{x}}{\sigma}$ where $\bar{x}$ is the mean and σ the standard deviation.

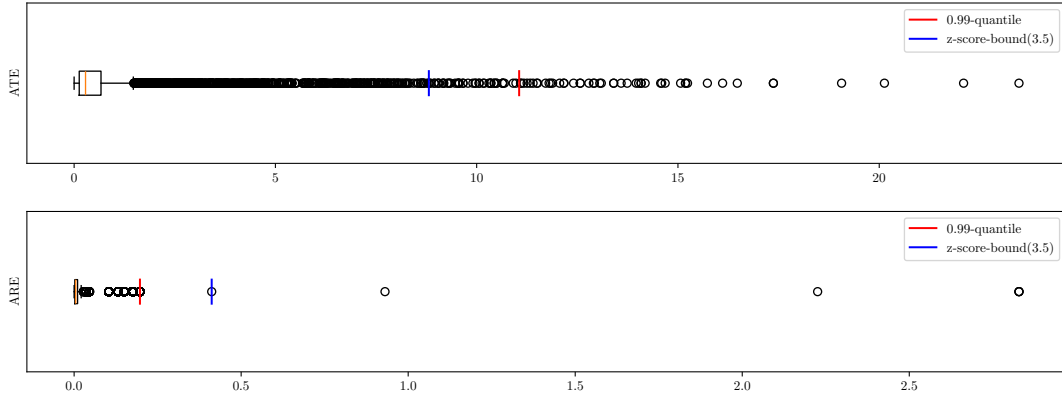


Figure 4: Box plot of the ATE (top) and ARE (bottom) values; is also shown the 0.99-quantile (red line) and the first value with a z-score equal to 3.5 (blue line).

To facilitate our models to learn each quantity, the following strategies have been applied:

- Outliers cleaning: data points with ATE or ARE values above the 0.99-quantile or with a Z-score greater than or equal to 3.5 have been removed to reduce the impact of extreme outliers. 113 datapoints were removed;
- Logarithmic rescaling: due to the skewed distribution of ATE and ARE, a logarithmic transformation has been applied to these values to reduce the effect of large values and improve model learning; let x be the interested value to be rescaled, the applied operation is:

$$\log(1 + x)$$

where $\log$ is the natural logarithm;

- Rescaling: after the logarithmic rescaling, a linear rescaling to $[0, 1]$ has been applied; let $\{x_1, \dots, x_n\}$ be interested values to be rescaled, the applied operation is

$$\frac{x_i}{\max_{1 \leq k \leq n} x_k} \quad i=1, \dots, n$$

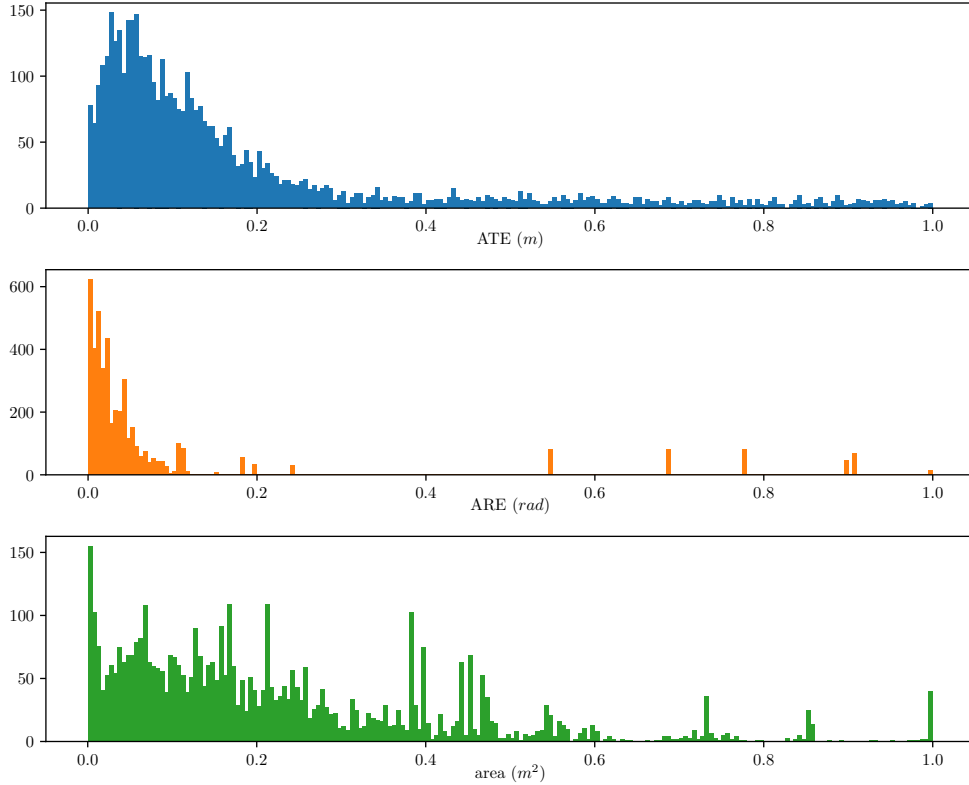


Figure 5: Dataset values after the log and linear rescaling.

- Image cropping: each map has been cropped to remove unnecessary borders, focusing only on the relevant area to reduce input size and eliminate irrelevant background; the crop is always squared in order to maintain this property to all floorplan images. Figure 6 show an example of this operation;
- Image resizing: each floorplan image is resized to 400×400 pixels to ensure a uniform input size for the neural network models;



Figure 6: A floorplan (left) and its bounding square crop (right).

2.3 Data augmentation

Given the relatively limited size of the dataset, the following data augmentation techniques have been applied to training set:

- Random flip: both horizontal and vertical flips applied;
- Random rotation: a random rotation between $0^\circ, 90^\circ, 180^\circ, 270^\circ$ is applied;

3 Model

In this project, different architectures are used, all based on Convolutional Neural Network (CNN) in order to delegate the feature extraction to the model and directly using the environment floorplan image. Formally, the model h we want to learn can be expressed as follows:

$$h : [0, 255]^{n \times n} \times \mathbb{R}^+ \rightarrow \mathbb{R}^2$$

Where $[0, 255]^{n \times n}$ is a floorplan image, encoded as a $n \times n$ grayscale image, $\mathbb{R}^+$ is the area of the floorplan expressed in m^2 and $\mathbb{R}^2$ contains ATE (m) and ARE (rad).

4 AlexNet

References

- [1] Rainer Kümmerle et al. “On Measuring the Accuracy of SLAM Algorithms”. In: *Autonomous Robots* 27 (Nov. 2009), pp. 387–407. DOI: 10.1007/s10514-009-9155-6.
- [2] Matteo Luperto and Francesco Amigoni. “Extracting Structure of Buildings Using Layout Reconstruction”. In: *Intelligent Autonomous Systems 15*. Ed. by Marcus Strand et al. Cham: Springer International Publishing, 2019, pp. 652–667. ISBN: 978-3-030-01370-7.
- [3] Matteo Luperto, Valerio Castelli, and Francesco Amigoni. “Predicting Performance of SLAM Algorithms”. In: *CoRR* abs/2109.02329 (2021). arXiv: 2109.02329. URL: <https://arxiv.org/abs/2109.02329>.
- [4] Cyrill Stachniss, John J. Leonard, and Sebastian Thrun. “Simultaneous Localization and Mapping”. In: *Springer Handbook of Robotics*. Ed. by Bruno Siciliano and Oussama Khatib. Cham: Springer International Publishing, 2016, pp. 1153–1176. ISBN: 978-3-319-32552-1. DOI: 10.1007/978-3-319-32552-1_46. URL: https://doi.org/10.1007/978-3-319-32552-1_46.